SISTEMAS OPERATIVOS

Aula Teórica 13

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Deadlocks

1. Resource
2. Introduction to deadlocks
3. The ostrich algorithm
4. Deadlock detection and recovery
5. Deadlock avoidance
6. Deadlock prevention
7. Other issues

Detection with Multiple Resource of Each Type (1)

When multiple copies of some of the resources exist we need a matrix based algorithm for detecting deadlock among n processes, P_1 through P_n. Let the number of resource classes be m, with E_1 resources of class 1, E_2 resources of class 2, and generally , E_i resources of class i ($1 \le i \le m$) . E is the **existing resource vector**. It gives the total number of instances of each resource in existance.
We need also two arrays, C, the **current allocation matrix**, and R the **request matrix**.

At any instant, some of the resources are assigned and are not available. Let A be the **available resource vector**, with A giving the number of instances of resource i that are currently available (not assigned).

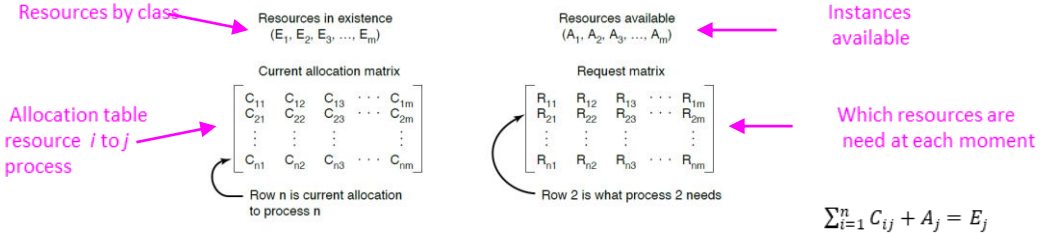


Figure 6-9. The four data structures needed by the deadlock detection algorithm.

Detection with Multiple Resource of Each Type (2)

Algorithm
The deadlock detection algorithm is based on comparing vectors.
Let us define the relation $A \leq B$, two vectors, where each element of A is less than or equal to the corresponding element of B , $A_i \leq B_i$ for $1 \leq i \leq m$.

Each process is initially said to be unmarked. As the algorithm progresses processes will be marked, indicating that they are able to complete and thus not dealocked.
When the algorithm terminates, any unmarked processes are known to be deadlocked.

Do
1. Look for an unmarked process, P_i, for which the ith row of R is less than equal to A.
2. If such a process is found, add the ith row of C to A, mark the process, and go back to step 1.
3. If no such process exists, the algorithm terminates.

End Do

What the algorithm are doing in step 1 is looking for a process that can be run to completion.
Such a process is characterized as having resources demands that can be met by the currently available resources.
The selected process is then run until it finishes. Concluding it will be returning all his resources (holded moments ago) to the pool of available resources.

If all the processes are ultimately able to run to completion, none of them are deadlocked.

Detection with Multiple Resource of Each Type (3)

Example:

Here we have 3 processes and 4 resouce classes, has shonw in the figure.

Process 1 has one scanner. Process 2 has two tape drivers and one camera. Process 3 has one plotter and two scanners.

Each process needs additional resources, as shown by matrix *R*.

Init. all processes are unmarked.

First. Look for a process whose resource request can be satisfied.

P_1 cannot be satisfeid because there isn't a camera available.

P_2 cannot be satisfeid because there isn't a scanner available.

P_3, $R_{P_3} \leq A$, runs and eventually returns all its resources, giving:

$$A=(2\ 2\ 2\ 0)$$

Next, P_2 can run and return its resources,

$$A=(4\ 2\ 2\ 1)$$

Now the remaning process P_1 can run.

There is no deadlock in the system.

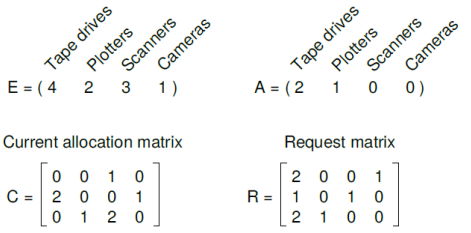


Figure 6-10. An example for the deadlock detection algorithm.

Consider a minor variation, suppose P_3 needs a camera as well, R_{P3}=(2 1 0 1), none of the requests can be satisfied so the entire system will eventually be deadlocked.

Recovery from Deadlock (1)

Now that we know how to detect deadlocks (at least with static resource request known in advance), the question is **when look for them comes up?**

- to check every time a resource request is made -> potentially expensive in terms of CPU;
- check every K seconds;
- or when the CPU utilization has dropped below some threshold.

Recovery through Preemption

- In some cases, it may be possible to temporarily take a resource away from its current owner and give it to another process.
- This is highly dependent on the nature of the resource.

Example: Laser printer.
To take a laser printer printing someones job, the job could be suspend (marked as not runnable), the operator could collect all the sheet already printed. At this point the printer can be assigned to another process. When this process finishes, the original process restarted.

Recovery from Deadlock (2)

Recovery through Rollback
Checkpoint a process periodically, writes its state to a file so that it can be restarted later if it is found deadlocked.

To be effective, new checkpoints should not overwrite old ones but should be written to new files.

Recovery through Killing Processes
The crudest but simplest way to break a deadlock is to kill one or more of the processes.

- in the deadlock cycle. With a little luck, the other processes will be able to continue.

- a process not in the cycle, must be carefully chosen because it is holding resources that some process in the cycle needs.

- It is best to kill a process that can be rerun from the beginning with no ill effect.
  Example: A compilation can always be rerun because all it does is read a source file and produce an object file.

- Never kill a process that updates databases, because it cannot always be run a second time safely.

Deadlock Avoidance (1)

In the discussion of deadlock detection , we tacitly assumed that when a process asks for resources, it asks for them all at once (the *R* matrix). In most systems, however, resources are requested one at a time. The system must be able decide whether granting a resource is safe or not and make the allocation only if it is safe.

Resource Trajectories

The main algorithms for deadlock avoidance are based on the concept of safe states.

First state point: *p*
A runs between *p* and *q*
B runs between *q* and *r*
A runs between *r* and *s*
When *A* crosses I_1 line , it requests, and is granted the printer
B runs between *s* and *t*
When *B* reaches point *t,* it requests then plotter

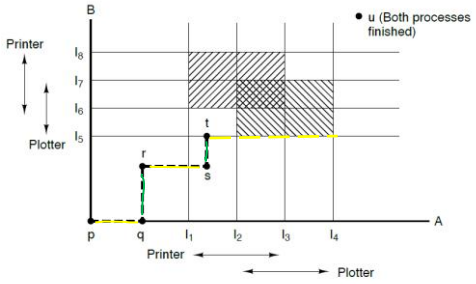


Figure 6-11. Two process resource trajectories.

Both shaded regions represents regions where both processes have the same resource (printer, plotter).
Mutual exclusion rule makes it impossible to enter in these areas.
If the system ever enter in the shaded area, it will eventually deadlock.

At point *t* the **only safe thing to do is run process *A* until it gets I_4**. Beyond that, any trajectory to *u* will do.

Deadlock Avoidance (2)

Safe and Unsafe States

The deadlock avoidance algorithm uses at any instant of time, the states of E, A, C and R.

A state is said to be **safe** if there is some scheduling order in which every process can run to completion even if all of them suddenly request their maximum number of resources immediatly.

Example 1:

We have a total of 10 instances of the resource, so with 7 resources already allocated (6-12 (a)) , three are still free.

- The state of (a) is safe because there exist a sequence of allocations that allows all processes to complete execution.
- Schedular can simply run *B* exclusively, it asks for two more instances of the resource leading to state (b).

-When *B* completes we get (c).

- Then the schedular can run C, leading eventually to (d), When C completes, we get (e).
- Now A can get the six instances of the resource it needs and also completes. So state (e) is also safe, can avoid deadlock.

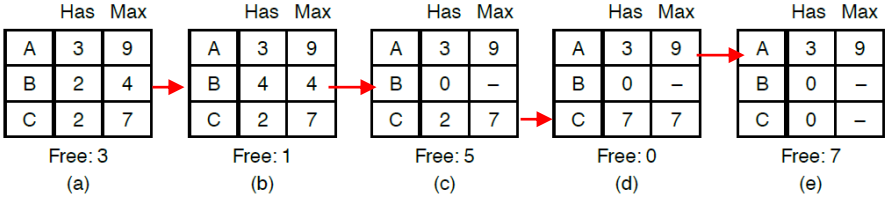


Figure 6-12. Demonstration that the state in (a) is safe.

Deadlock Avoidance (3)

Example 2:

We have a total of 10 instances of the resource, so with 7 resources already allocated (6-13 (a)) , three are still free.

- This time *A* requests a resource and it is granted (b).
- Can we find a sequence that is guaranted to work? Schedular can run *B* exclusively, it asks for two more instances of the resource leading to state (c).
- When *B* completes we get (d).
- Now there is no sequence that guarantees completion.

The allocation decision that moved the system from (a) to (b), went from a safe state for an unsafe state.

It is worth noting that an unsafe state it is not a deadlock state. It is possible that *A* might release a resource before asking for any more, allowing *C* to complete and avoiding deadlock altogether.

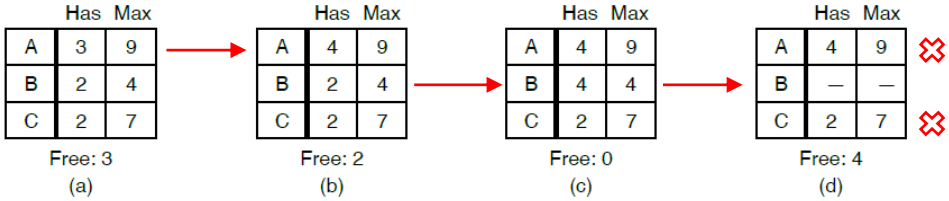


Figure 6-13. Demonstration that the state in (b) is not safe.

Deadlock Avoidance (4)

Remenbering: It is worth noting that an unsafe state it is not a deadlock state.

The difference between a safe state and an unsafe state is that :

- from a safe state the system can guarantee that all processes will finish;
- from a unsafe state, no such guarantee can be given.

The Banker's Algorithm for a single Resource
A scheduling algorithm that can avoid deadlocks is due to Dijkstra (1965).
It is modeled on the way of a small-town banker might deal with his customers to whom he has granted lines of credit (banks did not lend money unless they knew they could be repaid).

- the algorithm checks to see if granting the request leads to an unsafe state?
  If so the request is denied.

Otherwise, if granting the request leads to a safe state. It is carried out.

Deadlock Avoidance (5)

Example

Four customers, A, B, C, and D, each of whom has been granted a certain number of credit units (e.g. 1 unit is 1K dollars). The banker knows that not all customers will need their maximum credit immediately, so he has reserved only 10 units rather than 22 to service them.

Customers make loans from time to time (analogy to asking for resources) and we get at a certain moment Fig.6-14(b). This state is safe because with two units left, the banker can delay any request except *C*'s thus letting *C* finish and release all four of his resources. With four units in hand, the banker can let either *D* or *B* have the necessary units , and so on.

To see if a state is safe, the banker ckecks to see if he has enough resources to satisfy a customer. If so, those loans are assumed to be repaid, and the customer now closest to the limit is checked, and so on. If all loans can eventually be repaid, the state is safe and the initial request can be granted.

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

Figure 6-14. Three resource allocation states: (a) safe. (b) safe. (c) unsafe.

The Banker's Algorithm for Multiple Resources

The matrix on the left shows how many of each resources are currently assigned to each of the five processes.
The matrix on the right shows how many of each resources each process still needs in order to complete.
Vector *E* – existing resources. Vector *P* – possessed resources. Vector *A* – available resources.

1. Look for a row, *R*, whose unmet resource needs are all smaller than or equal to *A*. If no such row exists, the system will eventually deadlock since no process can run to completion (assuming processes keep all resources until they exit).
2. Assume the process of the chosen row request all the resources it needs (which is grantred toi be possible) and finishes. Mark that process as terminated and add of its resources to the *A* vector.
3. Repeat step 1 and 2 until either all processes are marked terminated (in which case the initial state is safe) or no process is left whose resource needs can be met (in which case the system was not safe).

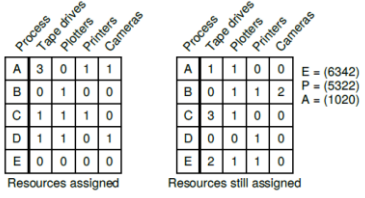


Figure 6-15. The banker's algorithm with multiple resources.

Deadlock Avoidance (7)

Tanenbaum and Bos, point out that although in theory the Banker's algorithm is wonderful, in pratice it is essentially useless because

- processes rarely know in advance what their maximum resource needs will be;

- the number of processes is not fixed, but dynamically varying as new users log in nand out;

- resources that were thought to be available can suddenly vanish (tape drives can break);

Few, if any, existing systems use Banker's algorithm to prevent deadlocks.

Some systems, however, use heuristics similar to those of Banker's algorithm to prevent deadlocks.

Deadlock Prevention (1)

Having seen that deadlock avoidance is essentially impossible, because it requires information from the future, which is not known, we will try to ensure that at least one of the four conditions stated to have a deadlock is never satisfied, then deadlocks will be structurally impossible.

Attacking the Mutual-Exclusion Condition

- For data, the simples method is to make data read only, so that processes can use the data concurrently;

- Some devices (such as printer) can be spooled to permit several processes to generate output at the same time.
  In this model, the only process that actually requests the physical printer is the printer deamon.
  Since the printer deamon never requests any other resources, deadlock for the printer is eliminated.

  However, if two processes each filled up one half of the available spooling space with output and neither was able to finished producing its full output, we would have two processes that had each fineshed part, but not all, of their output, and could not continue. Neither process will ever finish, so we have a deadlock on the disk.
  Not all devices can be spooled – Principle.

<< Avoid assigning resources when not absolutely necessary >>
<< Try to make sure that as few processes as possible actually claim the resource >>

Deadlock Prevention (2)

Attacking the Holt-and-Wait Condition
If we can prevent processes that hold resources from waiting for more resources, we can eliminate deadlocks.

-Require all processes to request all their resources before starting execution;

If everything is available, the process will be allocated whatever it needs and run to completion.

If one or more resources are busy, nothing will be allocated and the process will just wait.

Problem
- Many processes do not know how many resources they will need until they started runnig;
- Resources will be not used optimally.

A slightly diferente way to break the hold-and-wait condition is to require a process requesting a resource to

first temporarily release all the resources it currently holds. Then it tries to get everthing it needs all at one.

Deadlock Prevention (3)

Attacking the Non-Preemption Condition

If a process has been assigned the printer and is in the middle of printing output, forcibly taking away the printer because a needed plotter is not available is tricky at best and impossible at worst.

However, somo resources can be virtualized to avoid this situation. Spooling printer output to the SSD or disk and allowing only the printer deamon access to the real printer eliminates deadlocks involving the printer, altough it creates a potential for deadlocks over disk space. With large SSDs/disks tought, running ou of storage space is unlikely.



Not all resources can be virtualized like this.

Deadlock Prevention (4)

Attacking the Circular Wait Condition
The circular wait can be eliminated in several ways.

- One way is simply to have a rule saying that a process is entitled only to a single resource at any moment. If it needs a second one, it must release the first one.

Example: In a large job printing, this restriction is unacceptable.

- Provide a global numbering to all the resources. Now the rule is this: processes can request resources whenever they want to, but all requests must be made in numerical order.

Example: In figure 6-16 a process may request a printer and then a tape drive, but it may not request first a plotter and then a printer.

Deadlock Prevention (5)

For any $i \neq j$ a deadlock occurs, if *A* and *B* could freely request any resource.

With the numerical order, deadlock it is impossible for $i > j$ or $i < j$.

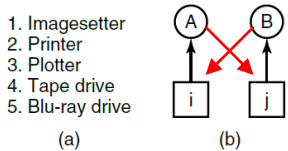


Figure 6-16. (a) Numerically ordered resources. (b) A resource graph.

Although numerically ordering the resources eliminates the problem of deadlocks, it may be impossible to find a ordering that satisfies everyone.

When the resources include process-table slots, disk spooler space, locked database records, and other abstract resources, the number of potential resources and different uses may be so large that no ordering could possible work.

Deadlock Prevention (5)

Summary of approaches to deadlock prevention

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Other Issues (1) - Two-Phase Locking

Although both avoidance and prevention are not terribly promising in the general case, for specific applications, many excellent special-purpose algorithms are known.

Example: In many database systems, an operation that occurs frequently is requesting locks on several records and then updating all the locked records. When multiple processes are running at the same time, there is a real danger of deadlocks.

Two-Phase Locking
Phase One
The process tries to lock all the records it needs, one at a time.
(No real work is done in phase one)
If needed records found locked, just release all its locks, waits a bit , start over;
If phase one succeeds, it starts second phase, performing updates and releasing the locks.

Note similarity to requesting all resources at once.

However, this strategy is not applicable in general. In real-time systems and process control systems, it is not acceptable to just terminate a process partway through because a resource is not available and start all over again. The algorithm works only in those situations where the programmer has very carefully arrange things so that the program can be stopped at any point during the first phase and restarted.

Other Issues (2) - Nonresource Deadlocks

Sometimes the resources are hardware or software objects, such as cameras or database records, but another times they are more abstract.

Resource deadlocks is a problem of **competition synchronization**.

Independent processes would complete service if their execution were not interleaved with competing processes. Interleaved access to locked resources, however, enables resource deadlock.

Possible for two processes to deadlock

- each is waiting for the other to do some task

Can happen with semaphores

- each process requires to do a *down()* on two semaphores (mutex and another) , Fig 6-5.

- if done in the wrong order, deadlock results.

Other Issues (3) – Livelock and Starvation

Livelock

In some situations, a process tries to be polite by giving up locks it already acquired whenever it notices it cannot obtain the next lock it needs. Then it waits around a millisecond, and tries again.

Now imagine a pair of processes running in parallel (perhaps on different cores) that use two resources, doing the same polite policy, at the same time. Both gets the first resource and when they found that cannot get the second one, they politely give up the lock they are currently holding, and start again. This repeats forever.

Clearly, no process is blocked, and we could even say that things are happening, still no progress is possible, so we do have something equivalent to deadlock, **livelock**.

Starvation

In a dynamic system , requests for resources happen all the time, so a policy is needed to make the decision about who gets which resource first. As an example consider the allocation of a printer. A possible allocation algorithm is to give to the process with the smallest file to print (shortest job) first to the printer,

Works great for multiple short jobs in a system

May cause long job to be postponed indefinitely even though not blocked

Solution: First-come, first-serve policy

Question:

What is the key difference between: Deadlock, Lifelock and Starvation? Give examples.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

